

Introduction to Databases

Autumn 2025

Exercise 4

(Theory) Hand-in: 19.11.2025 (ADAM, 23:59)
(Practical) Hand-in: 19.11.2025 (during exercise)

Solving the Exercises: The exercises can be solved in small groups of a maximum of two people. Use the notations introduced in the lecture. The DMI plagiarism guidelines apply for this lecture.

Submission Information: Please upload all (Theory) deliverables BEFORE the deadline to ADAM as a **single PDF**. For (Practical) exercise check the **Hand-In** instructions. The solutions must contain your names. Solutions that are handed in too late cannot be considered.

Task (Practical) 1: A day in the life of a SQL statement (3 Points)

You are given the following SQL statement:

```
SELECT canton.name, COUNT(village.name) AS villages
FROM canton, village
WHERE village.canton_id = canton.id
GROUP BY canton.name
HAVING COUNT(village.name) > 20
ORDER BY canton.name
```

What does this query do? What is the value of first column of the first row?

Your job is to visualize the different steps of the query. Starting with the tables canton and village and ending with the results, show the different operations that are applied in between and which part of the SQL statement they correspond to.

Hand-In: Show your visualization to one of the assistants/tutors and explain it.

Task (Theory) 2: SQL (6 Points)

Answer the each of the following questions about the chess database with a *single* SQL statement, unless mentioned otherwise.

You can check some of your SQL statements by executing them in the SQLite database from last week. Be careful: Your SQL statements must comply with the dialect from the lecture! SQLite might accept SQL statements that are invalid according to the lecture and vice versa.

In your queries state all **JOIN** types explicitly. Using ***** in **COUNT** is forbidden.

1. What is the **opening_text** of the opening with id 42?
2. How many players are in the database?
3. For each player id find the maximum **diff** over all their games.
4. For each player list the **player_id**, **game_id** and **diff** where **diff** is equal to the maximum over all their games.
5. For each **opening_text** show how often it was played.
6. Which player has won the most games? Sort players in descending order by the number of games they have won. Remember that “1-0” means that white has won.
7. For each player (**player_name**) count how many different kinds of openings they have played. List the top ten players with different openings.
8. Find all pairs of players that have played against each other more than 100 times (not counting player playing against themselves). Order the results from high to low.
9. List for each **game_id** the name of the event and the tournament URL (or NULL if there is none).
10. Find all players which have won exactly the same number of times playing white and playing black. For each of them list their **player_id** and how often they have won as white.
11. Add a column to **titles** called **title_long** of type TEXT.
12. Fill the new **title_long** column for the existing titles. This requires more than one statement.

Hand-In: Insert your statements into the template provided on ADAM.

Task (Theory) 3: Joins & Cross Product (1 Points)

customers	(<u>customer_id</u> , customer_name)
orders	(<u>orders_id</u> , order_date, <u>customer_id</u>)

You are presented with the following SQL query, which uses a **INNER JOIN** to link the **orders** and the **customers** tables:

```
SELECT
    orders.order_id,
    orders.order_date,
    customers.customer_name
FROM
    customers
INNER JOIN
    orders ON customers.customer_id = orders.customer_id;
WHERE order_date > '2025-07-01'
```

Adjust this query so it does no longer use a **JOIN** operator but still retrieves the same tuples.

Hand-In: The new query and a short explanation (max. 2 sentences). Put your solution in the same file you used for task 2.